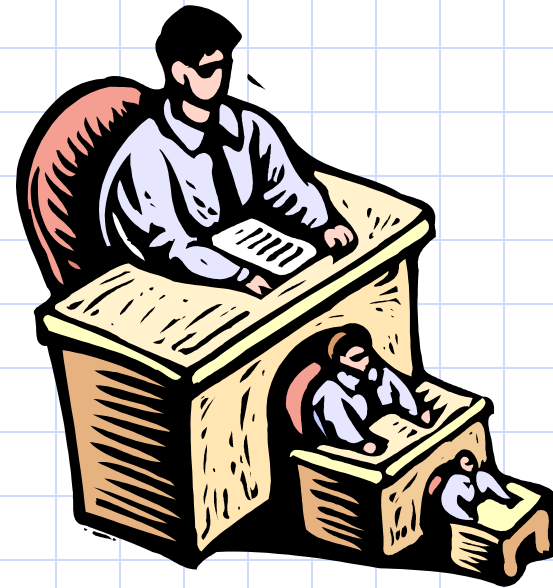


Recursion and binary search



The Recursion Pattern

- **Recursion:** when a method calls itself
- Classic example--the factorial function:
 - $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot (n-1) \cdot n$
- Recursive definition:

$$f(n) = \begin{cases} 1 & \text{if } n=0 \\ n \cdot f(n-1) & \text{else} \end{cases}$$

- As a Python method:

```
1 def factorial(n):  
2     if n == 0:  
3         return 1  
4     else:  
5         return n * factorial(n-1)
```

Content of a Recursive Method

- **Base case(s)**

- Values of the input variables for which we perform no recursive calls are called **base cases** (there should be at least one base case).
- Every possible chain of recursive calls **must** eventually reach a base case.

- **Recursive calls**

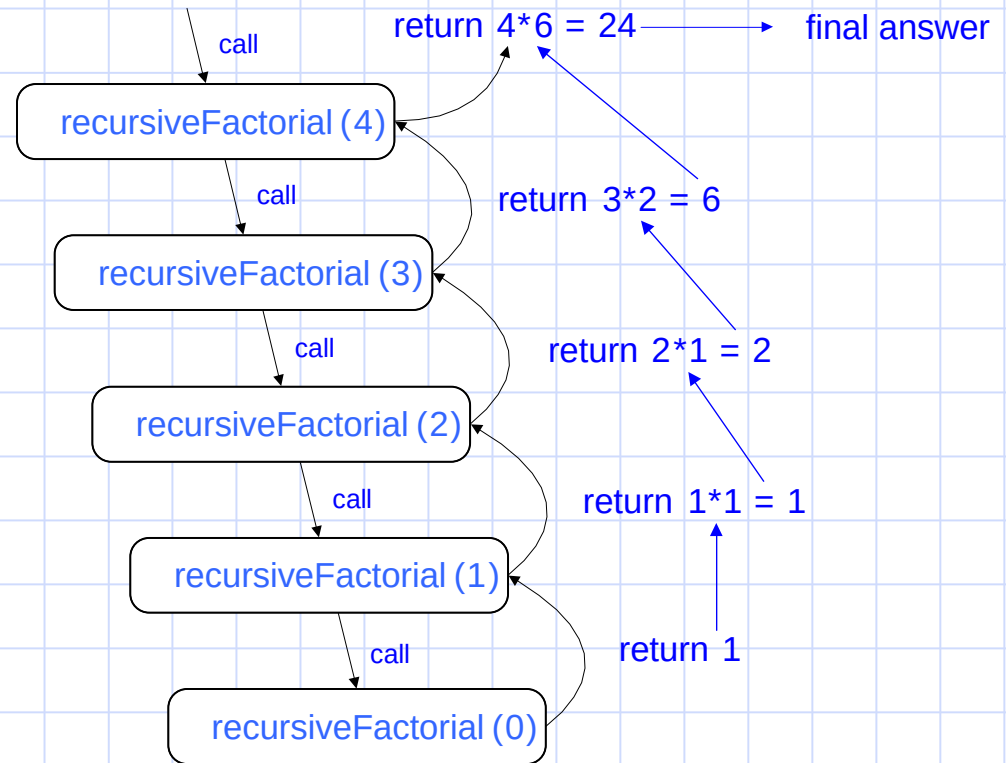
- Calls to the current method.
- Each recursive call should be defined so that it makes progress towards a base case.

Visualizing Recursion

Recursion trace

- A box for each recursive call
- An arrow from each caller to callee
- An arrow from each callee to caller showing return value

Example



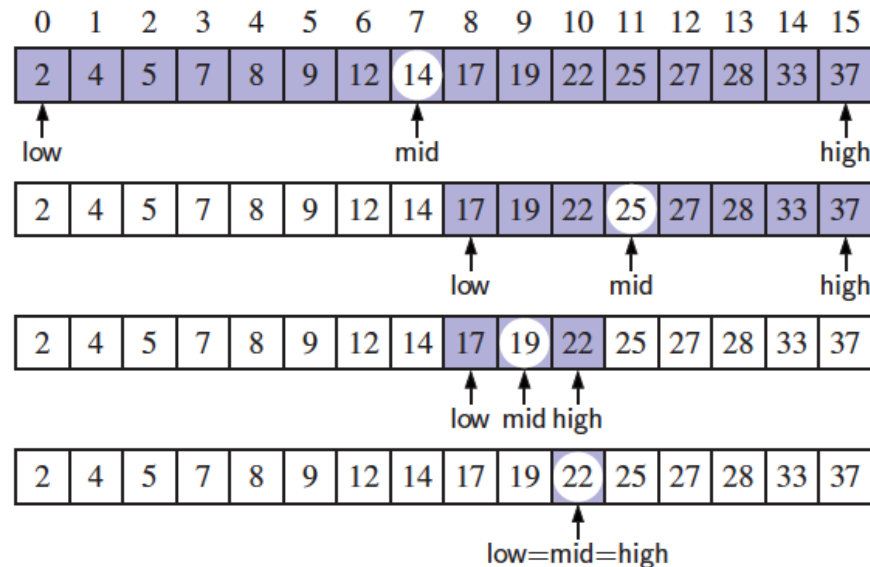
Binary Search

- Search for an integer, target, in an ordered list.

```
1 def binary_search(data, target, low, high):
2     """ Return True if target is found in indicated portion of a Python list.
3
4     The search only considers the portion from data[low] to data[high] inclusive.
5     """
6     if low > high:
7         return False                                # interval is empty; no match
8     else:
9         mid = (low + high) // 2
10        if target == data[mid]:                      # found a match
11            return True
12        elif target < data[mid]:
13            # recur on the portion left of the middle
14            return binary_search(data, target, low, mid - 1)
15        else:
16            # recur on the portion right of the middle
17            return binary_search(data, target, mid + 1, high)
```

Visualizing Binary Search

- We consider three cases:
 - If the target equals $\text{data}[\text{mid}]$, then we have found the target.
 - If $\text{target} < \text{data}[\text{mid}]$, then we recur on the first half of the sequence.
 - If $\text{target} > \text{data}[\text{mid}]$, then we recur on the second half of the seq



Analyzing Binary Search

- Runs in $O(\log n)$ time.
 - The remaining portion of the list is of size $\text{high} - \text{low} + 1$.
 - After one comparison, this becomes one of the following:

$$(\text{mid} - 1) - \text{low} + 1 = \left\lfloor \frac{\text{low} + \text{high}}{2} \right\rfloor - \text{low} \leq \frac{\text{high} - \text{low} + 1}{2}$$

$$\text{high} - (\text{mid} + 1) + 1 = \text{high} - \left\lfloor \frac{\text{low} + \text{high}}{2} \right\rfloor \leq \frac{\text{high} - \text{low} + 1}{2}.$$

- Thus, each recursive call divides the search region in half; hence, there can be at most $\log n$ levels.